

速習 コレクション

list / dict / tuple

rkskmt

1

コレクション (Collection) とは

- 複数のデータを塊として管理するためのデータ型
→ コンテナとも呼ばれる

CC-BY-SA 4.0:

https://en.wikipedia.org/wiki/Collecting#/media/File:Numismatic_items_or_specimens_on_display_at_an_exhib



2

よく使われるCollection型

この3種類だけは絶対に知っておく必要あり

型	特徴
<i>list</i>	最もよく使う。順番あり・変更可
<i>dict</i>	キーで管理。名前でアクセス可
<i>tuple</i>	要素固定。変更不可

他にもあるが、この三種だけは使い方も含め絶対に知っておくこと

3

一般的にはコンテナとも呼ばれる

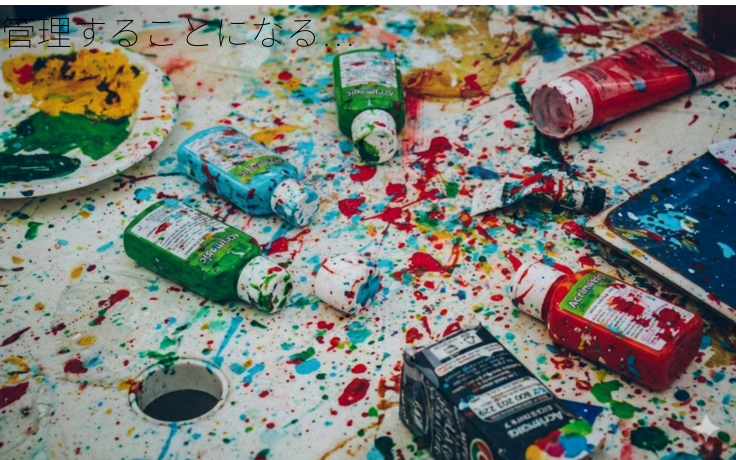
- 英単語の意味：集めること・収集
→ Contain（内に含む）+ er（やつ）



4

もしコレクション型がなかったら

大量のデータを1つずつ変数で管理することになる。



5

クラスの定期テスト成績を処理したい

データの特徴：1クラスあたり30数十のレコード。学生番号・名前・各教科の成績で構成。成績の判定や平均点などを出したい。

生徒番号	名前	国語	数学	英語
1	佐藤 太郎	85	90	85
2	鈴木 花子	90	85	80
3	高橋 一郎	78	95	92
4	田中 次郎	92	70	68
5	伊藤 三郎	88	88	90
6	渡辺 四郎	78	82	80
7	山本 五郎	92	88	91
8	中村 六子	70	75	72
9	小林 七子	88	90	85
10	加藤 八郎	80	85	83
...	...	...	...	...

(悪い例) コレクション型なし

変数を1つずつ作るとどうなるか？

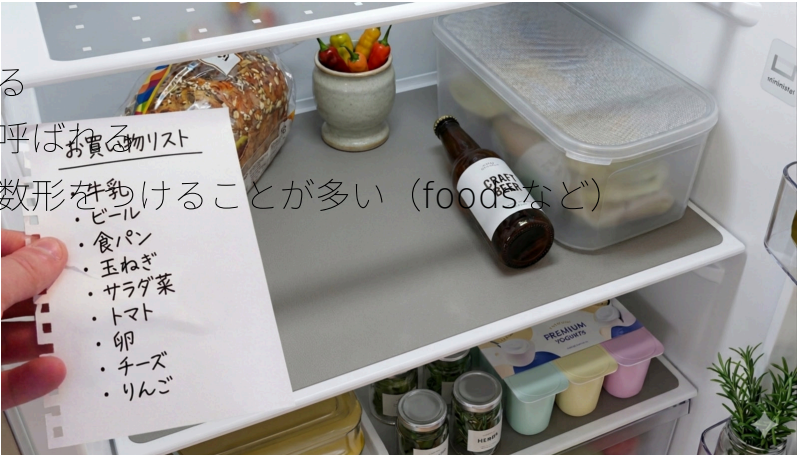
```
1 R06_0001 = 85 # 令和6年度 学生番号1番
2 R06_0002 = 90
3 R06_0003 = 78
4 R06_0004 = 92
5 R06_0005 = 88
6
7 # 合格者の判定
8 if R06_0001 >= 90:
9     print("学生R06_0001はSです")
10 if R06_0002 >= 90:
11     print("学生R06_0002はSです")
12
13 # 以下、R06_0003以降も同様にしていく??? (非効率すぎる)
```

❶ コレクション型を使えば...

for 文で一気に処理できる。学生が30人いても同じコードで対応可能。

【Collection 1】 list (リスト型)

- 最もよく使われるコンテナ
- データは追加順に並んでいる
- 他言語では配列やアレイと呼ばれる
- 変数名は、要素の名詞の複数形を付けることが多い (foods など)



list の初期化

- まずは、右辺で **list 型ですよ** と宣言しないと始まらない
- ケースにより色々あるが **[]** が一般的

```
1
2 # 空のリスト (初期化) を作りたいときは以下のどちらか
3 empty1 = []
4 empty2 = list()
5
6 # カンマ区切りで初期値付きの初期化
7 scores = [85, 90, 78, 92, 88]
8
9 # print文で全体を表示
10 print(scores) # [85, 90, 78, 92, 88]
11 print(empty1) # []
12 print(empty2) # []
13
14 # rangeから変換して初期化
15 odd_numbers = list(range(1, 11, 2)) # [1, 3, 5, 7, 9]
```

アクセス方法

- **[]** に数字（インデックス or 添字）でアクセス
→ **[1]**、**[500]**
- インデックスは先頭が **0**（ゼロ）
→ なので、思ったより 1 つ少ない数でアクセス
 - ▶ 5 番目の要素にアクセスしたい → **[4]**

```
1 print(scores[0])    # 85
2 print(scores[2])    # 78
3
4 # リストの長さ以上のインデックスを指定するとエラー
5 #print(scores[5])    # IndexError: list index out of range
6 # ↑ 長さは5だが、ゼロ始まりなので4が末尾
7
8 # マイナスを指定すると末尾からのアクセスになる
9 # ただし、リストの末尾は -0 ではなく -1
10 print(scores[-1])   # 88
11 print(scores[-5])   # 85
12
13 # リストの長さ+1のマイナスを指定するとエラー
14 print(scores[-6])   # IndexError: list index out of range
```

10

要素の追加（append）と結合

```
1 # 要素の追加は .append を使う（末尾に追加）
2 scores.append(95)
3 print(scores)    # [85, 90, 78, 92, 88, 95]
4
5 # 【注意】このような書き方はできない
6 score2 = scores.append(60)    # エラーにはならないが...
7 print(score2)    # None ← リストではない！（60自体は追加されている）
8
9 # リスト同士を接続（concat）することも可能
10 classA_scores = [72, 81]
11 classB_scores = [94, 66]
12 other_scores = classA_scores + classB_scores
13 print(other_scores)    # [72, 81, 94, 66]
14
15 # これでも可
16 scores += other_scores
17 print(scores)    # [85, 90, 78, 92, 88, 95, 60, 72, 81, 94, 66]
```

11

すべての要素へのアクセス（走査）

- 基本的に **for in** で使われる

```
1 # ① 基本系
2 for score in scores:
3     print(f"{score}点")
4
5 # ② これも稀に使う
6 for i in range(len(scores)):
7     print(f"学生{i + 1}は{scores[i]}点")
8
9 # ③ enumerate で ↑を同時に実現
10 for i, score in enumerate(scores):
11     print(f"学生{i + 1}は{score}点")
```

💡 どれを使う？

- 要素だけ必要 → ①
- 番号も必要 → ③（**enumerate** が一番スッキリ）
- スライスなど index 操作が必要 → ②

12

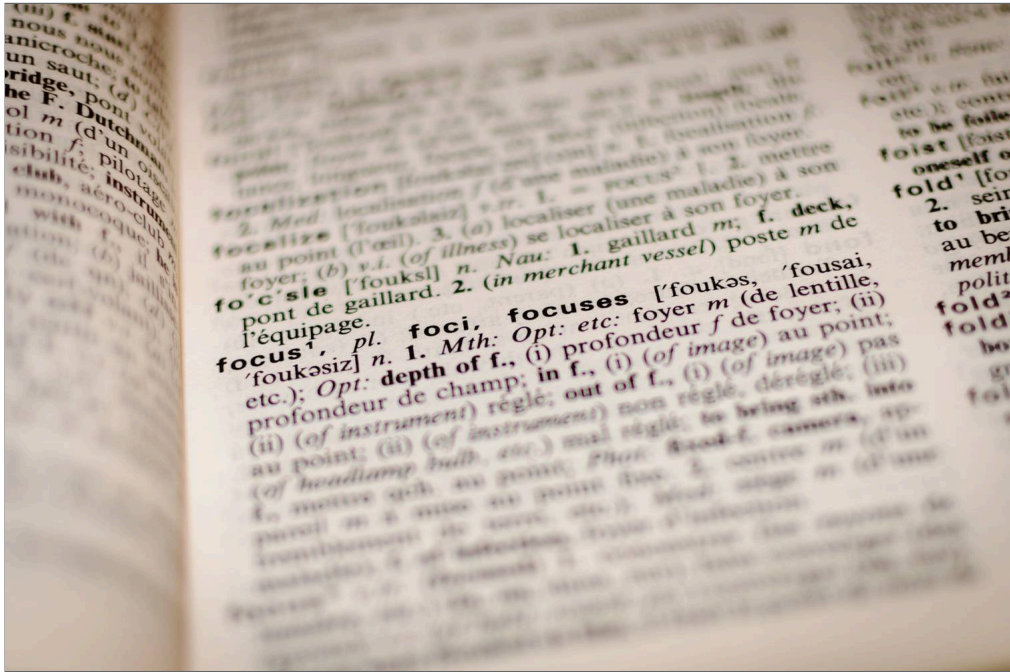
演習問題（list）



```
1 student_scores = [85, 90, 92, 78, 92, 88]
```

- 1. **append** を使って **100点** の学生を追加しよう
- 2. **90点以上** の学生の「学生番号」と「点数」を表示するプログラムを書こう
→ ヒント：**for** 文の中で **if** 文を使う
- 3. **平均点**を表示しよう
▶ 解答を見る

【Collection 2】 dict（辞書型）



CC0:
[https://en.wikipedia.org/wiki/Fictionary#/media/File:Focus_definition_\(Unsplash\).jpg](https://en.wikipedia.org/wiki/Fictionary#/media/File:Focus_definition_(Unsplash).jpg)

dict の初期化

- **{}** に **キー：値** のペアをカンマ区切りで書いて初期化
- 変数名は **name2score** のような「**x2y**」形式が慣例
→ **2** は「→ (to)」の意味
▶ 「A2B」は「A to B」で「AからBへ変換」

```
1 # {"キー1": 値1, "キー2": 値2, ...} のように初期化
2 name2score = {"Yamada": 95, "Suzuki": 75}
3
4 # print で表示可能
5 print(name2score)          # {'Yamada': 95, 'Suzuki': 75}
6 print(type(name2score))    # <class 'dict'>
7
8 # 空の dict の初期化
9 dict_test1 = {}
10 dict_test2 = dict()
```

要素の追加・アクセス方法

- **["キー"]** でアクセス（list の **[0]** に相当）
- 存在しないキーへの代入 → **追加**、存在するキーへの代入 → **上書き**

```
1 # ["キー"] でアクセス
2 print(name2score["Yamada"]) # 95
3
4 # 存在しないキーを指定するとエラー
5 print(name2score["Takahashi"]) # KeyError: 'Takahashi'
6
7 # 存在しないキーに代入すると追加
8 name2score["Takahashi"] = 80
9 print(name2score) # {'Yamada': 95, 'Suzuki': 75, 'Takahashi': 80}
10
11 # 存在するキーに代入すると上書き
12 name2score["Suzuki"] = 70
13 print(name2score) # {'Yamada': 95, 'Suzuki': 70, 'Takahashi': 80}
```

キーの存在確認（**in** 演算子）

in 演算子でキーや値の存在を確認できる。存在しないキーを指定する前に使うと **KeyError** を防げる。

```
1 name2score = {"Yamada": 95, "Suzuki": 75}
2
3 # キーが存在するか確認
4 if "Yamada" in name2score:
5     print(name2score["Yamada"]) # 95
6
7 # 存在しないキーも安全にアクセスできる
8 if "Tanaka" in name2score:
9     print(name2score["Tanaka"])
10 else:
11     print("Tanaka は登録されていません")
12
13 # not in で「存在しない」を確認
14 if "Sato" not in name2score:
15     print("Sato は未登録")
```

17

for 文での使い方（**dict**）

- **.keys()** / **.values()** / **.items()** の3種類を使い分ける
- 実務では **.items()** が最頻出

```
1 # すべてのキーを列挙
2 print(name2score.keys()) # dict_keys(['Yamada', 'Suzuki', 'Takahashi'])
3
4 # すべての値を列挙
5 print(name2score.values()) # dict_values([95, 70, 80])
6
7 # キーと値の組を列挙
8 print(name2score.items()) # dict_items([('Yamada', 95), ...])
9
10 # キーだけ使うパターン
11 for key in name2score.keys():
12     print(f"{key}さんが居ます")
13
14 # キーと値を同時に使うパターン（最もよく使う）
15 for key, value in name2score.items():
16     print(f"{key}さんは{value}点です")
```

18

演習問題（**dict**）



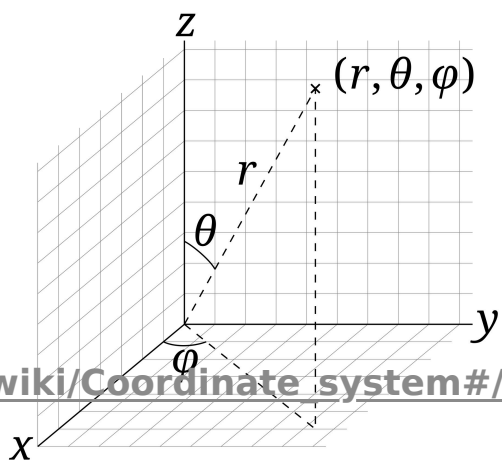
```
1 name2score = {"Yamada": 95, "Suzuki": 75, "Takahashi": 80}
```

1. **dict** に100点を取った **“Tanaka”** さんを追加しよう
2. **Takahashi** さんの点数を **92点** に訂正しよう
3. **90点以上** の学生の「名前」と「点数」を表示するプログラムを書こう
→ ヒント：**items()** を使って value を判定
4. **"Yamada"** が **dict** に存在するか **in** を使って確認し、存在すれば点数を表示しよう
▶ 解答を見る

19

【Collection 3】 **tuple** (タプル)

- 組のこと。
→ 5つ組み等
- 数学用語からきている
→ 例：3-tuple、n-tuple
- 要素が固定のリスト
- **()** で書かれる



PD: https://en.wikipedia.org/wiki/Coordinate_system#/media/File:3D_Spherical.svg

20

数学用語からきている

- **tuple** は、**組**のこと
- よく似た概念に**集合**（**set**）がある

性質	tuple （組）	set （集合）
順序	あり $(a, b) \neq (b, a)$	なし $\{a, b\} = \{b, a\}$
重複	区別する $(1, 1, 2) \neq (1, 2)$	区別しない $\{1, 1, 2\} = \{1, 2\}$
定義	デカルト積（n個の集合の直積）	要素の集まり
用途	座標・関数の引数・関係	包含・和・積

💡 Pythonでの **tuple** と **set**

- **tuple** → 順序あり・重複あり・変更不可
- **set** → 順序なし・重複なし（数学の集合に近い）

tuple の使いどころ

- 要素数が固定のデータを表すのに使う
 - 座標 **(x, y)**、RGB **(r, g, b)** など
 - **list** より **メモリ効率がよい**
- 値の変更ができない（**イミュータブル**）
 - 意図せぬ書き換えを防げる
- 関数から複数の値を返すときによく使う

tuple の初期化・アクセス

- **()** にカンマ区切りで値を書いて初期化
 - 空の初期化もありうるが大抵は意味がない
- アクセス方法は **list** と同じ（インデックス指定）

```
1 # 英語・数学・理科の点数を tuple で表現
2 score = (90, 85, 72)
3 print(score)           # (90, 85, 72)
4 print(type(score))     # <class 'tuple'>
5
6 # アクセスは list と同じ
7 print(score[0])        # 90
8 print(score[-1])       # 72 (末尾からもOK)
9 print(score[3])        # IndexError: tuple index out of range
```

Python

tuple と **list** の違い・制限

```
1 # 値の変更はエラー (listでは可能。タプルでは付加)
2 score[0] = 100 # TypeError: 'tuple' object does not support item assignment
3
4 # 値を追加できない (同じく)
5 score.append(100) # error
6
7 # どうしても追加したい場合は、list へ一度変換すれば可能
8 # しかし、普通は使わない
9 score_list = list(score)
10 print(type(score_list)) # <class 'list'>
11 score_list.append(100)
12 score = tuple(score_list)
13 print(score_list)      # [90, 85, 72, 100]
```

Python

💡 なぜ **tuple** を使う？

変更させたくないデータを保護できる。**list** よりメモリ効率もよい。

よくある **tuple** の使われ方（関数の戻り値）

```
1 # リストの最小値と最大値を返す関数
2 def get_min_max_value(data):
3     return (min(data), max(data)) # ← tuple で複数の値を返す
4
5 scores = [85, 90, 78, 92, 88]
6 answer = get_min_max_value(scores)
7 print(answer) # (78, 92)
8
9 # 以下のように分けて受けることも可能
10 min_val, max_val = get_min_max_value(scores)
```

💡 アンパック（unpacking）

tuple などの **イテラブル** なクラスの各要素を複数の変数に一度に代入することを **アンパック** という。

```
1 point = (3, 7)
2 x, y = point      # x=3, y=7
```

変数の数と要素数が一致しないとエラーになる。

① イテラブル（iterable）とは

イテレーション（**iteration**）は「コンテナから要素を1つずつ順番に取り出す操作」のこと。どの言語でも **for** 文の基本動作はこれ。イテラブルとは、イテレーションができるもの＝順番に要素を取り出せるもの。

型	イテラブル？
list	☐
tuple	☐
str	☐ （1文字ずつ）
range	☐
dict	☐ （キーが順に出る）
int / float	☒

アンパックも **for** 文も、イテラブルであれば同じように使える。

実は **dict** で出てきていた **tuple**

dict のメソッド **.items()** はタプルの **list** を返している。だから **key, value** という2変数に代入できる。

```
1 print(name2score.items())
2 # dict_items([('Yamada', 95), ('Suzuki', 70), ('Takahashi', 80)])
3 #          ^^^^^^^^^^^^^^^^^ ← これが tuple!
4
5 for key, value in name2score.items(): # tuple を2変数に展開
6     print(f"{key}さんは{value}点です")
```

演習問題（**tuple**）



```
1 def get_min_max(scores):
2     return min(scores), max(scores) # カッコなしでも tuple になる!
3
4 scores = [85, 90, 78, 92, 88]
```

- 関数を引数scoresでコールして、戻り値を変数 **result** で受け取る
- type(result)** と値を **print** する
- low, high** の2変数でアンパックして「最低: ○点, 最高: ○点」と表示
- result[0] = 100** と改ざんを試してみよう
▶ 解答を見る

冒頭の成績表、いまなら持てる

- **list** と **dict** を組み合わせると、最初に見た成績表がそのままコードになる

```
1 students = [  
2     {"番号": 1, "名前": "佐藤 太郎", "国語": 85, "数学": 90, "英語": 85},  
3     {"番号": 2, "名前": "鈴木 花子", "国語": 90, "数学": 85, "英語": 80},  
4     {"番号": 3, "名前": "高橋 一郎", "国語": 78, "数学": 95, "英語": 92},  
5 ]  
6  
7 # 全員の3教科平均を一気に計算（30人でも300人でも同じコード）  
8 for s in students:  
9     avg = (s["国語"] + s["数学"] + s["英語"]) / 3  
10    print(f'{s["名前"]}: 平均 {avg:.1f}点')  
11 # 佐藤 太郎: 平均 86.7点  
12 # 鈴木 花子: 平均 85.0点  
13 # 高橋 一郎: 平均 88.3点
```

- 「変数を1人ずつ作る」世界には、もう戻らなくていい

まとめ

コレクションは複数のデータをまとめて管理するための型。

型	宣言	アクセス	追加・変更	主な用途
list	[1, 2, 3]	[i]	○	順番あり・汎用
dict	{"k": v}	["key"]	○	名前でアクセス
tuple	(1, 2, 3)	[i]	✕	座標・戻り値

発展：その他のコレクション型

現場でも競技プログラミングでもよく使われる。各自で調べよう。

- **deque** — 先端への挿入・削除が高速（**double ended queue**）
- **defaultdict** — キーが存在しない場合にデフォルト値を返す **dict**
- **heapq** — 最小値の取り出しに強い（heap木で管理）
- **set** — 重複が許されない。和集合などの論理演算も可能

```
1 A = {'a', 'b', 'c'}  
2 B = {'b', 'c', 'd'}  
3 print(A | B) # {'a', 'b', 'c', 'd'} ← 和集合  
4 print(A & B) # {'b', 'c'}          ← 積集合  
5 print(A ^ B) # {'a', 'd'}          ← 対称差  
6 print(A - B) # {'a'}               ← 差集合
```

数値データには **numpy** の **ndarray** がよく使われる（後日扱う）。